Entrega 1: manejo de GitHub y Docker

Hugo Alexis Fonseca Díaz

Politécnico Grancolombiano Institución Universitaria

Énfasis Profesional I (Integración Continua)

Edward Reyes Corredor

8 de noviembre de 2026

Entrega 1: manejo de GitHub y Docker

La integración continua consiste en que los integrantes de un equipo incorporen sus cambios al código compartido con frecuencia y en que cada cambio sea verificado de forma automática (Fowler, 2006; Oliveros Acosta, s.f.-a). Para que esta práctica sea posible, el código debe alojarse en un repositorio centralizado con control de versiones (Oliveros Acosta, s.f.-b), y el ambiente de ejecución debe poder reproducirse en cualquier equipo (Oliveros Acosta, s.f.-c).

Este documento corresponde a la primera entrega del proyecto del módulo. Su objetivo es alojar el proyecto en GitHub y usar Docker para construir dos contenedores que se comunican entre sí.

Marco conceptual

Máquinas virtuales y contenedores

Una máquina virtual ejecuta un sistema operativo completo sobre un hipervisor. Un contenedor comparte el kernel del sistema anfitrión y no requiere un sistema operativo independiente, por lo que es más liviano y rápido de desplegar (Oliveros Acosta, s.f.-c).

Docker y Docker Compose

Docker empaqueta una aplicación con sus dependencias en una imagen, que se describe en un Dockerfile (Docker Inc., s.f.-b). Docker Compose define varios servicios, sus redes y sus dependencias en un solo archivo YAML, y los levanta con un comando (Docker Inc., s.f.-a). Dentro de una misma red, cada servicio puede encontrarse por su nombre (Docker Inc., s.f.-c).

GitHub y control de versiones

Git guarda el historial de cambios del proyecto como instantáneas del repositorio en el tiempo, y GitHub aloja repositorios Git en la nube (GitHub, Inc., s.f.; Oliveros Acosta, s.f.-b).

Desarrollo del proyecto

Repositorio en GitHub

El proyecto está alojado en el repositorio https://github.com/halfonseca/Integraci-n-continua, dentro de la carpeta `Entrega 1`.

Arquitectura de los contenedores

La solución tiene dos servicios, descritos en la Tabla 1. Cada uno se construye a partir de su propio Dockerfile.

Tabla 1

Servicios del proyecto

Servicio	Imagen base	Puerto	Función
web	`python:3.12-slim`	5000 (publicado)	Aplicación Flask que registra cada visita
db	`postgres:16-alpine`	5432 (interno)	Base de datos PostgreSQL con la tabla `visitas`

La aplicación web se conecta a la base de datos usando el nombre del servicio, `db`, como host. Docker resuelve ese nombre dentro de la red `red_app`, por lo que no se usan direcciones IP fijas. El servicio `web` espera a que la base de datos responda al chequeo de salud con `pg_isready`. La base de datos no publica ningún puerto hacia el equipo anfitrión. El Listado 1 muestra la configuración completa.

Listado 1

docker-compose.yml

```
services:
  web:
    build: ./app
    container_name: web
    ports:
      - "5000:5000"
```

```
    environment:
      DB_HOST: db
      POSTGRES_DB: visitas
      POSTGRES_USER: usuario
      POSTGRES_PASSWORD: clave
    depends_on:
      db:
        condition: service_healthy
    networks:
      - red_app

  db:
    build: ./db
    container_name: db
    environment:
      POSTGRES_DB: visitas
      POSTGRES_USER: usuario
      POSTGRES_PASSWORD: clave
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U usuario -d visitas"]
      interval: 5s
      timeout: 3s
      retries: 10
    networks:
      - red_app

networks:
  red_app:
    driver: bridge
```

El Listado 2 muestra el Dockerfile del servicio web, que parte de una imagen oficial de Python y ejecuta la aplicación con el servidor de Flask (Pallets, s.f.). El Dockerfile de la base de datos parte de `postgres:16-alpine` y copia el archivo `init.sql` a la carpeta `/docker-entrypoint-initdb.d/`. PostgreSQL ejecuta los scripts de esa carpeta solo cuando la base de datos se crea por primera vez (Docker Inc., s.f.-d).

Listado 2

app/Dockerfile

```
FROM python:3.12-slim

ENV PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["flask", "--app", "app", "run", "--host=0.0.0.0", "--port=5000"]
```

Construcción y ejecución

Los dos contenedores se construyen y se levantan con el siguiente comando:

```
$ docker compose up --build -d
 Image proyecto-db Built
 Image proyecto-web Built
 Network proyecto_red_app Created
 Container db Healthy
 Container web Started
```

Las imágenes generadas se listan con `docker images`:

```
REPOSITORY     TAG          IMAGE ID       SIZE
proyecto-web   latest       b5a3ad9bddae   212MB
proyecto-db    latest       15bda521753e   419MB
postgres       16-alpine    aa6cb33cf4c1   420MB
```

El estado de los servicios se revisa con `docker compose ps`:

```
SERVICE   IMAGE          STATUS                    PORTS
db        proyecto-db    Up 2 minutes (healthy)    5432/tcp
```

```
web       proyecto-web   Up About a minute        0.0.0.0:5000->5000/tcp,
   [::]:5000->5000/tcp
```

Verificación de la comunicación

Primero se enviaron tres peticiones a la aplicación desde el equipo anfitrión:

```
$ curl http://localhost:5000/
Visitas registradas: 1
$ curl http://localhost:5000/
Visitas registradas: 2
$ curl http://localhost:5000/
Visitas registradas: 3
```

Cada petición llega al contenedor `web`, que escribe el registro en el contenedor `db` y devuelve el total. Después se resolvió el nombre `db` desde el contenedor `web`:

```
$ docker compose exec web getent hosts db
172.18.0.2      db
```

Por último, se consultó la tabla `visitas` desde el contenedor `db`:

```
$ docker compose exec db psql -U usuario -d visitas -c "SELECT id, fecha
   FROM visitas ORDER BY id;"
 id |           fecha
----+----------------------------
  1 | 2026-11-09 00:19:27.288742
  2 | 2026-11-09 00:19:27.304808
  3 | 2026-11-09 00:19:27.319316
(3 rows)
```

Las fechas corresponden al reloj del contenedor. Los tres registros confirman que la aplicación escribe en la base de datos a través de la red de Docker, y que los datos se conservan en el contenedor `db`.

Entorno de pruebas

La Tabla 2 resume las versiones de software utilizadas, incluida la biblioteca de conexión a PostgreSQL (Psycopg Team, s.f.).

Tabla 2

Versiones de software utilizadas

Herramienta	Versión
Docker Engine	29.9.3
Docker Compose	v5.7.0
Python	3.12.16
Flask	3.1.3
psycopg2	2.9.13
PostgreSQL	16.15

Conclusiones

Se cumplió el requisito de la Entrega 1. El proyecto está alojado en GitHub y los dos contenedores construidos con Docker se comunican entre sí: la aplicación web registra cada visita en PostgreSQL y los datos se conservan en el contenedor de la base de datos.

La solución tiene limitaciones que deben tenerse en cuenta. La aplicación usa el servidor de desarrollo de Flask, que no es adecuado para producción. Las credenciales están escritas en el archivo de Compose, lo que solo es aceptable en un entorno de desarrollo. Además, la imagen de PostgreSQL ocupa 420 MB, casi el doble de la imagen de la aplicación (212 MB).

Referencias

Docker Inc. (s.f.-a). *Docker Compose overview*. https://docs.docker.com/compose/

Docker Inc. (s.f.-b). *Dockerfile reference*. https://docs.docker.com/reference/dockerfile/

Docker Inc. (s.f.-c). *Networking in Compose*. https://docs.docker.com/compose/how-tos/networking/

Docker Inc. (s.f.-d). *postgres - Official Image*. https://hub.docker.com/_/postgres

Fowler, M. (2006). *Continuous integration*. https://martinfowler.com/articles/continuousIntegration.html

GitHub, Inc. (s.f.). *GitHub Docs*. https://docs.github.com/

Oliveros Acosta, E. E. (s.f.-a). *Fundamentos de la integración continua* [Lectura fundamental]. Politécnico Grancolombiano.

Oliveros Acosta, E. E. (s.f.-b). *Manejo de repositorios* [Lectura fundamental]. Politécnico Grancolombiano.

Oliveros Acosta, E. E. (s.f.-c). *Máquinas virtuales y Dockers para construcción de ambientes* [Lectura fundamental]. Politécnico Grancolombiano.

Pallets. (s.f.). *Flask documentation (3.1.x)*. https://flask.palletsprojects.com/

Psycopg Team. (s.f.). *Psycopg 2.9.13 documentation*. https://www.psycopg.org/docs/